

Aula 3 — Setup + Unitários + Integração

"Testa a lógica isolada. Depois testa as peças juntas."

Jest · React Native Testing Library · renderHook

Prof. Jackson Smith Moisés Matias

PUC Minas IEC · 22/06/2026

Recap da Aula 2

- **Fork + PR no GitHub** — como entregar atividades (fluxo completo, CI embutida nos PRs)
- **Pirâmide de testes** — por que mobile tem mais testes manuais (Khorikov)
- **Testes manuais estruturados**: FEW HICCUPPS (Bolton) + tours (Whittaker) + Charter/SBTM (Bach)
- **Exercício ao vivo**: Organic Maps — 3 heurísticas + Anti-Social Tour
- **Template de bug report mobile** — SO, device, build number, frequência de reprodução

Setup RN + Jest/RNTL ficou para hoje — começa agora.

Atividade 1 — prazo era 10/06. Entregue?

Agenda da aula (3h30)

1. **Bloco 1 (75min)** — Setup + testes unitários: funções puras, lógica de estado, cobertura
2. **Intervalo (30min)**
3. **Bloco 2 (75min)** — Testes de integração: RNTL com contexto, navegação, API mock, `renderHook`
4. **Esquentar (15min)** — Atividade 2 + prévia da Aula 4 (Maestro E2E)

Objetivos de aprendizagem

- **Configurar** o projeto RN e rodar a suíte de testes pela primeira vez
- **Implementar** testes de domain puro com Jest (funções puras e lógica de estado)
- **Implementar** testes de componentes com React Native Testing Library
- **Configurar** cobertura mínima via Istanbul + ler o relatório
- **Implementar** testes de integração: componente com navegação + API mockada
- **Usar** `renderHook` para testar hooks com dependências

Teste manual vs automatizado — em mobile

Na Aula 2 você fez **teste manual/exploratório**. Hoje começa o **automatizado**. Não competem — se completam.

	Manual / Exploratório (Aula 2)	Automatizado (de hoje em diante)
Forte em	usabilidade, UX, tela nova, bug visual	regressão, regra de negócio, fluxo repetido
Frequência	uma vez · exploração humana	toda vez · a cada PR no CI
Em mobile	gestos, sensores, permissões, device real	cobrir a fragmentação (muitos OS/devices)
Custo	tempo humano a cada rodada	escreve 1x, roda pra sempre

Automatize o **repetitivo e estável** (regressão); explore **manualmente** o novo e o subjetivo. A fragmentação mobile torna a automação essencial — mas gesto/sensor ainda pede mão humana.

O que a pesquisa recomenda (mobile)

Cruz, Abreu & Lo (2019) — *"Guess what, test your app!"* (Empirical Software Engineering)

Analysaram centenas de apps Android open-source: a **maioria tem pouco ou nenhum teste automatizado**. Os que testam evoluem com **menos bugs e regressões**.

Recomendação prática — converge com Khorikov e a pirâmide:

- Comece pela **lógica de negócio** (unit) — barato, rápido, alto retorno
- **Automatize a regressão** e rode no **CI** a cada mudança
- Reserve **E2E** (caro e frágil) só pros **fluxos críticos**

Linhares-Vásquez et al. (2017, ICSME) reforça: teste mobile deve ser **contínuo e integrado ao desenvolvimento** — não um esforço pontual no fim.

Glossário rápido — você vai ouvir isso hoje

Flaky

teste que ora passa, ora falha **sem mudar o código**. Falso alarme (tempo, animação, ordem, rede) — mina a confiança na suíte.

Renderizar

"desenhar" o componente **em memória** pra inspecioná-lo no teste, sem abrir o app no simulador.

Cobertura (coverage)

% do código que os testes executam. Cobertura alta $\neq$ bons testes.

Assertão (`expect`)

a verificação que faz o teste passar ou falhar: `expect(x).toBe(y)`.

Regressão

algo que funcionava e quebrou; testes de regressão pegam isso cedo.

Por que unit antes de integração?

E2E · Maestro ← Aula 4

Integração · RNTL ← Bloco 2 de hoje

Unitários · Jest ← Bloco 1 de hoje

Não dá pra depurar integração sem entender o unit. E não faz sentido E2E sem integração estável por baixo.

JS/TS puro vs React — dois mundos, dois custos

	JS / TS puro	React (RN)
O que é	lógica, dados, regras	componente que desenha UI
Exemplo	<code>isFavorito</code> , <code>posterUrl</code> , <code>favoritesStore</code>	<code><MovieCard/></code> , <code><MovieList/></code>
Tem tela?	não	sim (<code><View></code> , <code><Text></code>)
Pra testar	Jest — só Node	RNTL — renderiza em memória
Tempo por teste	~1 ms	~dezenas de ms
Flaky?	praticamente 0	pode flakar (render/async)

A régua é o **custo de render**: **lógica** → **Jest (sem tela)**; **componente** → **RNTL (com tela)**. E2E só na Aula 4 (segundos por teste). Regra de negócio mora no **puro**; UI é a casca.

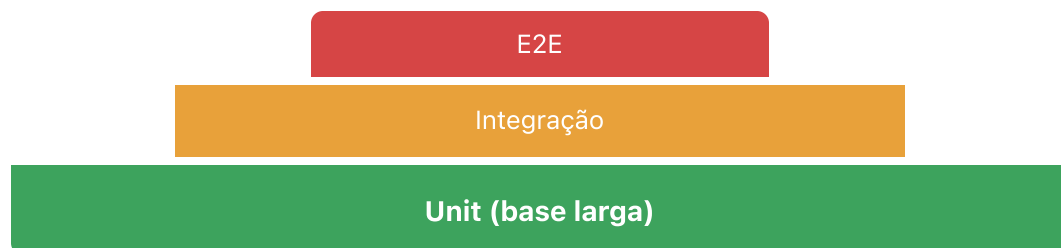
Mapa do dia — o que testamos e com quê

Quando	O que testar	Ferramenta	Render?
Bloco 1 (unit)	<code>posterUrl</code> , <code>isFavorito</code> , <code>favoritesStore</code> , <code>counterStore</code>	Jest	não — só Node
Bloco 1 (unit)	<code>MovieCard</code>	RNTL	sim — em memória
Bloco 2 (integração)	<code>useFavorites</code> (hook)	renderHook	meio-termo
Bloco 2 (integração)	<code>MovieList</code> + navegação + API mockada	RNTL	sim — árvore real
Aula 4	app inteiro rodando no device	Maestro	app real

Mesma régua do slide anterior: **sem tela = Jest**, **com tela = RNTL**. `renderHook` é o meio-termo pros hooks (Bloco 2). Comece pela base (Jest) — barato e estável.

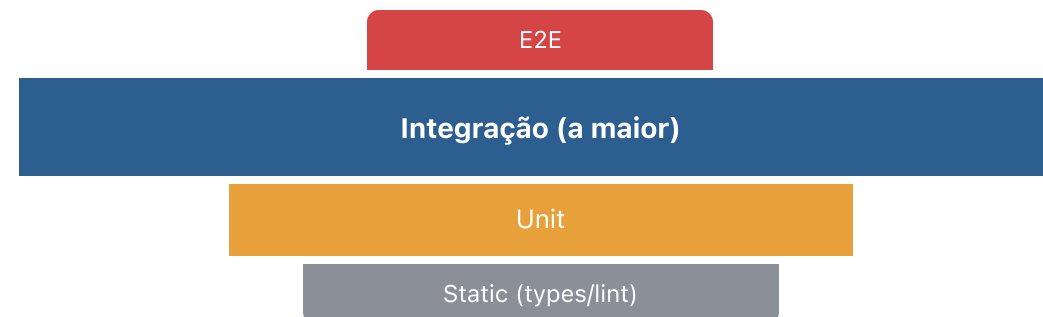
Pirâmide vs Troféu — conflitam?

Pirâmide



Cohn (2009) · Fowler
maximize **unit** isolado

Troféu



Kent C. Dodds (2018)
maximize **integração**

Não é guerra — é ênfase. Os dois cortam E2E (caro/frágil). A diferença é o meio: em **UI/RN o maior valor está na integração** (RNTL com deps reais). Dodds: *"quanto mais o teste se parece com o uso real, mais confiança ele dá"*. Por isso hoje fazemos **unit (Bloco 1) + integração (Bloco 2)** — E2E fica pra Aula 4.

Lógica pura — da história ao TypeScript

História: o ❤️ do filme fica **cheio** quando é favorito e **vazio** quando não é. Pra saber qual desenhar, o app faz uma pergunta simples: esse *filme está na lista de favoritos*?

Algoritmo (pense antes de abrir o editor):

- recebe a **lista de ids favoritados** e o **id do filme**
- responde **sim** se o id está na lista, **não** se não está

```
// regra pura – entra (lista, id), sai (sim/não). Sem React, sem tela.  
function isFavorito(ids: number[], id: number): boolean {  
  return ids.includes(id);  
}
```

Como testar — só pensar em entrada → saída:

- `isFavorito([1, 2, 3], 2) → true`
- `isFavorito([1, 2, 3], 9) → false`

Sem simulador, sem mock. Jest roda isso em < 1ms. É a base da `favoritesStore` (próximo slide) — lá o método se chama `isFavorite` — que você testa na Atividade 2.

Domain layer — Jest puro

Lógica de negócio isolada de React e do nativo. Teste mais barato e mais estável.

```
// src/utils/poster-url.ts
export const posterUrl = (
  path: string | null,
  size: 'w185' | 'w342' | 'w500' = 'w342'
) => (path ? `https://image.tmdb.org/t/p/${size}${path}` : null);
```

```
// __tests__/posterUrl.test.ts
import { posterUrl } from '../src/utils/poster-url';

describe('posterUrl', () => {
  it('monta a URL com o size padrão w342', () => {
    expect(posterUrl('/abc.jpg')).toBe('https://image.tmdb.org/t/p/w342/abc.jpg');
  });

  it('respeita o size informado', () => {
    expect(posterUrl('/abc.jpg', 'w500')).toBe('https://image.tmdb.org/t/p/w500/abc.jpg');
  });

  it('retorna null quando o path é null', () => {
    expect(posterUrl(null)).toBeNull();
  });
});
```

Domain layer — lógica de estado

A `favoritesStore` **já vem pronta** (é a `isFavorito` do slide anterior + estado) — trate como caixa-preta e teste o comportamento:

```
// src/store/favoritesStore.ts – o contrato (você só testa)
add(id)           // adiciona o id           toggle(id)       // alterna add/remove
remove(id)        // remove o id           isFavorite(id)    // → true / false
clear()           // esvazia a lista
```

```
// __tests__/favoritesStore.test.ts ← ISSO é o seu trabalho (Atividade 2)
import { useFavoritesStore } from '../favoritesStore';
const store = () => useFavoritesStore.getState();

beforeEach(() => store().clear()); // store é singleton: limpe entre testes

test('toggle alterna – adiciona e depois remove', () => {
  store().toggle(42); // ausente → adiciona
  expect(store().isFavorite(42)).toBe(true);
  store().toggle(42); // presente → remove
  expect(store().isFavorite(42)).toBe(false);
});
```

Test Doubles — os que você usa hoje (Meszaros, 2007)

Test double = objeto falso que substitui uma dependência real (rede, navegação, módulo nativo) pro teste não depender dela.

Tipo	O que faz	No app de filmes (Atividade 2)
Stub	Devolve resposta fixa	<code>jest.mock('@services/api')</code> → filmes falsos (<code>popularMovies</code>)
Spy	Grava as chamadas pra inspecionar	mock do <code>useNavigation</code> no <code>MovieCard</code>
Mock	Spy + expectativa: falha se não for chamado certo	ver exemplo ↓

```
// Mock do navigate: jest.fn() grava as chamadas (Spy)...
const navigate = jest.fn();
jest.mock('@react-navigation/native', () => ({ useNavigation: () => ({ navigate }) }));

fireEvent.press(screen.getByText('Matrix'));
// ...e a expectativa cobra a chamada certa (Mock):
expect(navigate).toHaveBeenCalledWith('Detail', { id: 42, title: 'Matrix' });
```

Dummy e *Fake* existem, mas são raros aqui. Nome vem do cinema: *stunt double* (dublê).

Mockando dependências — jest.mock

```
// src/queries/movies/get-popular-movies.ts
import { api } from '@services/api';

export const fetchPopularMovies = async (page = 1) => {
  const res = await api.get('/movie/popular', { params: { page } });
  return res.data;
};

// __tests__/popularMovies.test.ts
import { fetchPopularMovies } from '../src/queries/movies/get-popular-movies';
import { api } from '../src/services/api';

jest.mock('@services/api');
const mockedGet = api.get as jest.Mock;

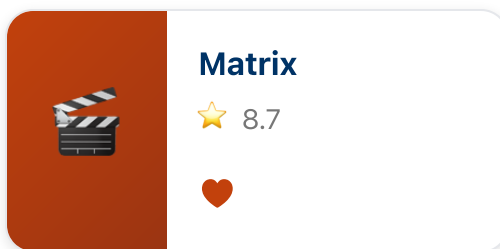
it('chama /movie/popular com a page e devolve o data', async () => {
  const resp = { page: 1, results: [{ id: 1, title: 'Matrix' }] };
  mockedGet.mockResolvedValue({ data: resp });

  const data = await fetchPopularMovies(1);

  expect(data).toEqual(resp);
  expect(mockedGet).toHaveBeenCalledWith('/movie/popular', { params: { page: 1 } });
});
```


Componente — da tela ao React

▢ O que o produto pediu



🔗 O componente React Native

```
<Pressable onPress={navigate}>
  <Image source={{ uri: posterUrl(...) }} />
  <View>
    <Text>{movie.title}</Text>
    <Text>★ {movie.vote_average}</Text>
  </View>
  <Text>♥</Text>
</Pressable>
```

Pressable → card clicável

Image → poster

Text → título e nota

♥ → botão favoritar

Mesma história de antes: o ♥ usa a lógica de **favoritos** (`isFavorito` / `favoritesStore`). React só **declara** o que aparece — o teste RNTL verifica sem simulador.

Componentes — React Native Testing Library

Testa comportamento de tela — o que o usuário vê e toca.

```
import { render, screen, fireEvent } from '@testing-library/react-native';
import MovieCard from '../src/components/MovieCard';

const mockNavigate = jest.fn();
jest.mock('@react-navigation/native', () => ({
  useNavigation: () => ({ navigate: mockNavigate }),
}));
const movie = { id: 42, title: 'Matrix', poster_path: '/m.jpg', vote_average: 8.7 };

it('mostra título e nota', () => {
  render(<MovieCard movie={movie} />);
  expect(screen.getByText('Matrix')).toBeTruthy();
  expect(screen.getByText('★ 8.7')).toBeTruthy();
});

it('navega ao tocar no card', () => {
  render(<MovieCard movie={movie} />);
  fireEvent.press(screen.getByText('Matrix'));
  expect(mockNavigate).toHaveBeenCalledWith('Detail', { id: 42, title: 'Matrix' });
});
```

RNTL — queries por prioridade

Query	Prioridade	Por quê
<code>getByRole</code>	✓ Alta	Reflete acessibilidade real
<code>getByLabelText</code>	✓ Alta	Segue <code>accessibilityLabel</code>
<code>getByText</code>	● Média	Frágil a mudanças de copy
<code>getById</code>	● Baixa	Escapa da UI real — última opção

```
// ✓ Prefira
screen.getByRole('button', { name: 'Adicionar favorito' })

// ● Evite como primeiro recurso
screen.getById('fav-button')
```

`testID` é escape hatch, não padrão — priorize queries semânticas. **Exceção:** itens dinâmicos por id (ex: `movie-card-1` na integração) e no E2E (Maestro), onde `testID` é o caminho estável.

Cobertura — Jest + Istanbul

```
npm test -- --coverage
open coverage/lcov-report/index.html
```

```
// jest.config.js - gate no CI
module.exports = {
  preset: 'jest-expo',
  collectCoverageFrom: ['src/**/*.{ts,tsx}', '!src/**/*.d.ts'],
  coverageThreshold: {
    global: { branches: 70, functions: 70, lines: 70, statements: 70 },
  },
};
```

Métrica	Mede
Statements	% de instruções executadas
Branches	% de caminhos (if/else , && , ?:)
Lines / Functions	% de linhas / funções chamadas

Lendo o relatório

Tests:	6 passed, 6 total				
-----	-----	-----	-----	-----	-----
File	% Stmt	% Brnc	% Func	% Line	Uncovered
-----	-----	-----	-----	-----	-----
favoritesStore.ts	100	50	100	100	12

- Branch 50% = você testou o toggle que adiciona, mas não o que remove → o caminho do remove ficou de fora
- Tests passed ≠ cobertura alta — são dimensões diferentes

! Dá pra ter 100 testes verdes cobrindo 10% do código. Cobrir os dois caminhos do toggle leva o branch a 100%.

Mutation testing com Stryker — cobertura mente

Cobertura diz "essa linha **rodou**" — não diz se o teste **pegaria um bug** nela.

Stryker muta seu código de propósito e roda a suíte:

```
if (score >= 8)    // seu código
if (score > 8)     // ← Stryker mutou. Algum teste falha?
```

- teste **falha** → mutante **morto**  (sua suíte pega o bug)
- teste **passa mesmo assim** → mutante **sobrevive**  (ponto cego)

Mutation score = % de mutantes mortos — qualidade real, não só execução.

```
npm run test:mutation    # roda DEPOIS de resolver os testes da store
```

100% de cobertura + mutation score baixo = ilusão. É o que fecha o Bloco 1.

O que faz um teste bom?

```
// ❌ sem assertion – cobre a linha, não prova nada  
it('renderiza', () => { render(<MovieCard movie={movie} />); });
```

```
// ✅ verifica o que o usuário vê  
it('exibe o título do filme', () => {  
  render(<MovieCard movie={movie} />);  
  expect(screen.getByText('Matrix')).toBeTruthy();  
});
```

```
// ❌ só verifica se o mock foi chamado – ignora o resultado  
expect(api.get).toHaveBeenCalled();
```

```
// ✅ verifica o que a função retornou de fato  
const data = await fetchPopularMovies(1);  
expect(data.results).toEqual([{ id: 1, title: 'Matrix' }]);  
expect(api.get).toHaveBeenCalledWith('/movie/popular', { params: { page: 1 } });
```

Teste bom: uma razão pra falhar, uma coisa verificada, nome que documenta a intenção.

Hands-on Bloco 1 — o app que vamos testar

Catálogo de filmes (RN + Expo). Implementado — você só escreve os testes.

```
cd puc-iec-testes-aplicacoes-mobile/exercicios/02-setup-suite-unitaria/exercicio
npm install && npm test # posterUrl já passa verde
```

Frentes de teste (sem nem rodar o app):

- `posterUrl` — helper que monta URL da imagem ← **começa aqui**
- `isTokenError` — classifica erro de autenticação da API
- `favoritesStore` · `counterStore` — lógica de estado (Zustand por baixo)
- **MovieCard (RNTL)** — renderiza e responde ao toque

Roteiro hands-on Bloco 1 (75min)

1. `npm test verde + ler posterUrl.test.ts` (modelo já pronto) — **5min**
2. `isTokenError` — função pura, escreve junto — **10min** ← confiança primeiro
3. `favoritesStore` — lógica de estado (add/remove/toggle) — **15min**
4. `MovieCard` (RNTL) — render do título/nota + `press` → navega — **20min** ★
5. `counterStore` — vocês sozinhos — **5min**
6. `npm run test:coverage` → ler relatório e discutir — **5min**
7. Mutation testing — Stryker demo rápida — **5min** (se sobrar tempo)

■ Bônus: `fetchPopularMovies` com `jest.mock` da api (+10min)

Pitfalls unit testing

Sleep em unit test — anti-pattern absoluto

Mock de tudo — acopla ao "como", não ao "o quê"

Nomes sem intenção (`testFunc1`) — revele o contrato

Coverage como métrica única — cobertura alta + mutation score baixo = ilusão

Teste sem `expect` — passa verde, não prova nada

Testar detalhes de implementação — quebra em refactor sem bug real

 **Intervalo — 30min**

Bloco 2 — Testes de Integração

Unitário testa peças isoladas. Integração testa **peças trabalhando juntas**.

	Unitário	Integração
O que isola?	tudo — moca dependências	só o que está fora do app (API, device)
Exemplo	favoritesStore sozinho	MovieList + store + navegação
Confiança	lógica interna	fluxo do ponto de vista do usuário
Velocidade	muito rápido	rápido (ainda sem simulador)

| RNTL é a ferramenta certa: renderiza a árvore real de componentes, mas sem simulador.

Integração: componente + navegação

Para testar componentes que usam `useNavigation`, envolva com `NavigationContainer`:

```
import { NavigationContainer } from '@react-navigation/native';
import { createNativeStackNavigator } from '@react-navigation/native-stack';
import { render, screen, fireEvent } from '@testing-library/react-native';
import MovieList from '../src/screens/MovieList';
import MovieDetail from '../src/screens/MovieDetail';

const Stack = createNativeStackNavigator();

function AppNavigator() {
  return (
    <NavigationContainer>
      <Stack.Navigator>
        <Stack.Screen name="MovieList" component={MovieList} />
        <Stack.Screen name="Detail" component={MovieDetail} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}

it('tap no card navega pra tela de detalhe', async () => {
  render(<AppNavigator />);
  fireEvent.press(await screen.findByText('Matrix'));
  expect(await screen.findByText('Detalhes do filme')).toBeTruthy();
});
```

Integração: componente + API mockada

Mocka a camada de dados — testa o componente com dados reais de formato:

```
import { QueryClient, QueryClientProvider } from '@tanstack/react-query';
import { render, screen } from '@testing-library/react-native';
import MovieList from '../src/screens/MovieList';
import * as movieService from '../src/services/movieService';

jest.mock('../src/services/movieService');
const mockedFetch = movieService.fetchPopularMovies as jest.Mock;

function wrapper({ children }: { children: React.ReactNode }) {
  return (
    <QueryClientProvider client={new QueryClient()}>
      {children}
    </QueryClientProvider>
  );
}

it('exibe lista de filmes retornada pela API', async () => {
  mockedFetch.mockResolvedValue([{ id: 1, title: 'Matrix', vote_average: 8.7 }]);
  render(<MovieList />, { wrapper });
  expect(await screen.findByText('Matrix')).toBeTruthy();
});
```

renderHook — testando hooks isolados

Quando o hook tem dependências (store, query, context) mas não quer renderizar a tela inteira:

```
import { renderHook, act } from '@testing-library/react-native';
import { useFavorites } from '../src/hooks/useFavorites';

it('toggle adiciona e remove favorito', () => {
  const { result } = renderHook(() => useFavorites());

  act(() => { result.current.toggle(42); });
  expect(result.current.isFavorite(42)).toBe(true);

  act(() => { result.current.toggle(42); });
  expect(result.current.isFavorite(42)).toBe(false);
});
```

`renderHook` é o meio-termo: roda o hook no ambiente React sem precisar de tela.

Integração: estado persistido entre telas

Testa fluxo completo dentro do React — favoritar em uma tela, verificar em outra:

```
it('favoritar em MovieList reflete no contador do header', async () => {  
  render(<AppNavigator />);  
  
  // Aguarda a lista carregar  
  const card = await screen.findByTestId('movie-card-1');  
  
  // Favorita  
  fireEvent.press(screen.getByTestId('movie-card-heart-1'));  
  
  // Verifica contador  
  expect(screen.getByTestId('favorites-count')).toHaveTextContent('1');  
});
```

Sem simulador, sem Maestro — só React renderizado em memória pelo RNTL.

Roteiro hands-on Bloco 2 (75min)

App: mesmo app do Bloco 1 — agora testando fluxos entre componentes.

1. `renderHook` no `useFavorites` — testar toggle isolado — **15min**
2. `MovieList` com `QueryClientProvider` + API mockada — lista aparece — **20min** ★
3. `MovieList` com `NavigationContainer` — tap navega pra Detail — **20min**
4. Integração completa: favoritar em List → contador atualiza — **15min**
5. Discutir: qual nível de teste cobriu o quê? — **5min**

Pitfalls testes de integração

Não limpar estado entre testes — Zustand persiste entre `it()` sem `beforeEach`

QueryClient compartilhado — crie um novo por teste (cache polui)

`findBy` vs `getBy` — use `findBy` (async) quando o dado vem de fetch

Mockar demais — store + nav + api + context mockados = não é integração

Ignorar `act()` — mutação de estado fora de `act()` gera warning e flake

Atividade 2 — revisão de prazo

A atividade 2 (suíte unitária) foi detalhada agora nesta aula. **Novo prazo: até 21/06.**

#	Entrega	Pontos
1	<code>favoritesStore</code> — 6 testes (add/remove/toggle/isFavorite/clear)	3
2	<code>MovieCard</code> (RNTL) — render + toque navega ★	3
3	<code>isTokenError</code> (data) — 5 testes	2
4	<code>counterStore</code> — 3 testes	1
5	Cobertura $\geq 70\%$ em <code>src/store</code> e <code>src/utills</code>	1
Eliminatório	<code>npm install && npm test</code> roda em $< 15\text{min}$	—
Bônus (+1)	<code>fetchPopularMovies</code> com <code>jest.mock</code> da api	+1

Esquenta — Aula 4: Maestro E2E

Na próxima aula vamos sair do RNTL (componentes em memória) e testar o **app real rodando no device**, com Maestro (YAML declarativo).

RNTL — hoje

render em memória
navegação mockada
API mockada
milissegundos por teste

Maestro — Aula 4

app real (simulador/emulador)
navegação real
rede real (ou interceptada)
segundos por teste

Pré-requisito para a Aula 4:

- Simulador iOS (macOS) **ou** emulador Android configurado
- Maestro instalado: `curl -Ls "https://get.maestro.mobile.dev" | bash`

Maestro é simples (YAML, sem build nativo) — instale antes pra ganhar tempo.

Leituras obrigatórias (pré-Aula 4)

- **Maestro Docs** — *Getting Started* (maestro.mobile.dev)
- **mobile.dev Blog** — *Why YAML for E2E Tests*
- **iDFlakies (Lam et al. 2019)** — catálogo de causas de flakiness em E2E

Próxima aula (22/06)

Aula 4 — E2E com Maestro

Do RNTL ao app real — YAML declarativo, conditional flows, fragmentos, Maestro Studio, Firebase Test Lab.

Pré-requisito:

- Atividade 2 entregue (21/06)
- Simulador iOS **ou** emulador Android funcionando
- `maestro --version` verde no terminal

Obrigado!

 jackson.96@gmail.com

 [linkedin.com/in/3jacksonsmith](https://www.linkedin.com/in/3jacksonsmith)

 github.com/jacksonsmith

Próxima segunda, 22/06, 19:00